

Problem 1.2 - Deterministic and Non-Deterministic MPI Output

Overview

The purpose of Problem 1.2 is to demonstrate an important difference between sequential and parallel programming. When multiple MPI processes execute concurrently, the order in which they reach an output statement is not inherently deterministic. Therefore, repeatedly executing the same parallel program can produce the same messages in different orders.

The objective was first to demonstrate this non-deterministic behavior using 16 MPI processes and then modify the program so that the processes always print their messages in sequential rank order: 0, 1, 2, ..., 15.

Environment Setup

The programs were executed on the Seawulf computing cluster using the short-28core partition.

1. Allocate Computing Resources

I allocated one node with 16 tasks and one CPU core per task:

```
salloc --partition=short-28core --nodes=1 --ntasks=16 --cpus-per-task=1
```

Here, `--ntasks=16` allocates resources for 16 MPI processes, while `--cpus-per-task=1` allocates one CPU core to each process.

2. Open an Interactive Shell

After the allocation was granted, I opened an interactive shell on the allocated compute node:

```
srun --pty bash
```

3. Load MPICH and GCC

The MPI implementation used for this problem was MPICH 3.4.2. Loading the MPICH module automatically loaded the compatible GCC compiler and other required dependencies:

```
module load mpich/gcc/3.4.2
```

The C programs were compiled using the MPI C compiler wrapper `mpicc`. For example:

```
mpicc hello_unordered.c -o hello_unordered
```

The `mpicc` wrapper invokes GCC while automatically supplying the headers and libraries required by MPICH.

Non-Deterministic MPI Program

The first program, `hello_unordered.c`, demonstrates the default non-deterministic behavior of parallel execution. It was executed using 16 MPI processes:

```
srun --mpi=pmi2 -n 16 ./hello_unordered
```

The `-n 16` option launches 16 copies of the program as separate MPI processes. Each process receives a unique MPI rank from 0 through 15. The `--mpi=pmi2` option allows the Slurm-launched processes to operate as members of the same MPI job.

Each process independently executes:

```
printf("Hello from Processor %d of %d\n", rank, size);
```

There is no synchronization specifying which rank should print first. Consequently, the output order depends on the timing of the concurrently executing processes and may change between executions.

The program was executed three times. The results are recorded in `log.txt`. Each run produced the same 16 messages but in a different ordering, demonstrating the non-deterministic behavior.

Deterministic MPI Program

The second program, `hello_ordered.c`, was designed to guarantee that the processes print in rank order from 0 through 15.

```
mpicc hello_ordered.c -o hello_ordered
srun --mpi=pmi2 -n 16 ./hello_ordered
```

The deterministic ordering was implemented using MPI point-to-point communication with `MPI_Send()` and `MPI_Recv()`.

Rank 0 prints first and then sends a small token to rank 1. Rank 1 waits at `MPI_Recv()` until it receives this token. It then prints its message and sends the token to rank 2. This process continues sequentially through all 16 ranks.

```
Rank 0 prints
|
v
Rank 1 prints
|
v
Rank 2 prints
|
v
...
|
v
Rank 15 prints
```

Although all 16 MPI processes are launched concurrently, each process after rank 0 is forced to wait for a message from the preceding rank before printing. This synchronization removes the non-determinism from the output order.

The deterministic program was also executed three times. The results are recorded in `ordered_log.txt`. Unlike the unordered program, all three executions produced the same processor ordering from rank 0 through rank 15.

Files

- `hello_unordered.c` - MPI program demonstrating non-deterministic output ordering.
- `hello_ordered.c` - MPI program using `MPI_Send()` and `MPI_Recv()` to enforce deterministic ordering.

- `log.txt` - Results from multiple executions of the unordered program.
- `ordered_log.txt` - Results from multiple executions of the ordered program.